18.REALM

22 March 2026 18:40

Knowledge captured during large model pretraining is stored in parameters
↳ this meant we need more larger networks to cover more facts

To capture knowledge in a more modular & interpretable way,
we augment LM pretraining with a latent knowledge retriever

Knowledge retriever → allows model to retrieve & attend over documents from a large corpus
↳ pretrain this in an unsupervised manner using Masked language modeling as a learning signal
and back propagating through a retrieval step that considers millions of documents

REALM → retrieval augmented language model pretraining
textual knowledge retriever

this approach asks the model to decide what knowledge to retrieve and use during inference

Retrieval improving model perplexity → rewarded
otherwise penalised

To address computation scale when retriever has to consider millions of docs
each computation → cached, asynchronously updated
→ selection of best docs can be formulated as Maximum Inner Product Search (MIPS)

REALM's $\underbrace{\text{pretraining} \rightarrow \text{fine tuning}}_{\text{retrieve then predict}}$

input x $p(y|x)$ for outputs y

Masked Language Modeling →
in pretraining x is a sentence from a corpus X with
some tokens masked out where the model must predict
those tokens y

REALM decomposes $p(y|x)$ → retrieve & predict
possibly helpful docs z from a knowledge corpus Z
$p(z|x)$ → This to predict output $p(y|z,x)$
we treat this as latent variable z and marginalize
over all possible documents z

$$p(y|z) = \sum_{z \in Z} p(y|z,x)\,p(z|x)$$

for MLM pre training task. we must predict the original value of
each [MASK] token in x

$$p(y|z,x) = \prod_{j=1}^{J_x} p(y_j|z,x)$$

$$p(y_j|z,x) \propto \exp\left(w_j^T \, BERT_{MASK(j)}(\text{join}_{BERT}(x, z_{body}))\right)$$

↓
Transformer output vector corresponding to the jth masked token
J_x is total no of [MASK] tokens in x
w_j is the learned word embedding for token y_j

Model Architecture

neural knowledge retriever $p(z|x)$
Knowledge-augmented encoder $p(y|z,x)$

$$p(z|x) = \frac{\exp f(x,z)}{\sum_{z'} \exp f(x,z')}$$

θ → all parameters associated with the retriever

$$f(x,z) = \text{Embed}_{input}(x)^T \, \text{Embed}_{doc}(z)$$

embedding functions that map x & z to d dimensional vectors
↳ inner product of vector embeddings

Tokens like in BERT paper
prefixing [CLS] token → Transformer produces a vector for this pooled representation of the sequence
[SEP] for separation
↳ also as final token

↓ linear projection to reduce the dimensionality of vector

$$\text{Embed}_{input}(x) = W_{input} \, BERT_{cls}(\text{join}_{BERT}(x))$$

$$\text{Embed}_{doc}(z) = W_{doc} \, BERT_{cls}(\text{join}_{BERT}(z_{title}, z_{body}))$$

$p(y|z,x)$ we join x & z into a single sentence to feed into a transformer (different one from retriever)

This allows rich cross attention between x & z before predicting y

Marginalization → soft aggregation over documents, not hard selection

Mechanistically Every doc contributes ... $p(z|x)$

J_x is total no of [mask] tokens in x

w_j is the learned word embedding for token y_j

Assuming answer 'y' to be found as a contiguous text in some doc z

Let $S(z,y)$ be a set of spans matching y in z

$$p(y|z,x) \propto \sum_{s \in S(z,y)} \exp\left(\underbrace{MLP}_{FFN}([h_{start(s)}, h_{end(s)}])\right)$$

$$h_{START(s)} = BERT_{START(s)}(\text{join}_{BERT}(x, z_{body}))$$

$$h_{END(s)} = BERT_{END(s)}(\text{join}_{BERT}(x, z_{body}))$$

transformer output vectors corresponding to the start & end tokens of span's

ϕ denotes all parameters associated with knowledge augmented encoder

This is a latent variable model

z = latent document

We never observe correct document

Softmax of inner product

$$f(x,z) = \text{Embed}_x(x)^T \text{Embed}_z(z)$$

this is similarity in embedding space

Retriever learns a space where query & relevant docs are close

Information flow

$x \to p(z|x) \to z \to p(y|z,x)$

with feedback

loss $\to$ retriever

Scaling issues

$\hookrightarrow$ training becomes slower

Compute $p(z|x) \propto \exp(f(x,z))$ over millions of docs

Cannot compute full softmax over all z

Need approximate retrieval

$\hookrightarrow$ this leads to MIPS, ANN

Bottleneck isn't combinatorial explosion but retrieval over large corpus

Retrieval = Soft Attention over documents

Transformer Att^n	REALM retrieval
attend over tokens	attend over docs
Softmax(q, K)	Softmax(x z)

It is not obvious how the training objective encourages meaningful retrievals

implicit supervision

Two learned spaces

$\to$ Query space

$\to$ Document space

Actually same vector space

$\to$ Input $\to$ BERT $\to$ token level outputs

$\downarrow$

Take [CLS] embedding

Then linear projection

$$\text{Embed}_x(w) = W\, BERT_{CLS}(x)$$

'unsure retriever $\to$ pretrained LM where knowledge stored in parameters

$\to$ knowledge update bottleneck

hallucinations

Why learning $p(z|x)$ slow?

problem Sum over many docs z But only few useful

Core issue

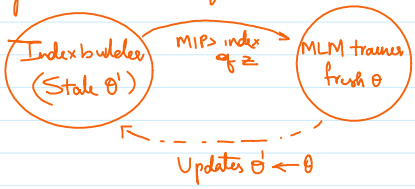
Early in training

$p(z|x)$ is almost uniform

Good docs get tiny probability

} diffuse probability mass

Asynchronous MIPS refreshes



Training objective

Maximize $\log p(y|x)$ approximated with top k

$$\log p(y|x) = \log \sum_z p(y|z,x)\, p(z|x)$$

Approximation

$$\simeq \log \sum_{z \in top\,k} p(y|z,x)\, p(z|x)$$

$p(y|z,x) \to$ soft aggregation over documents, not hard selection

Mechanistically Every doc contributes weighted by $p(z|x)$

Not pick best doc But "blend multiple docs probabilistically"

Retriever is rewarded for selecting correct document But it is not explicit

XNo explicit supervision

Instead

Loss = MLM loss on final prediction

Gradients flow

$y \to p(y|z,x) \to p(z|x)$

retriever is trained implicitly via downstream loss

Gradient flow $\to$ Strong gradients

$\to$ Generator (BERT encoder)

Weak/indirect gradients

$\to$ Retriever

which creates credit assignment problem across documents

They model

Answer = span inside document

predict $\to$ start position

$\to$ end position

Not free generation

MLM $\to$ pretraining

Span prediction $\to$ QA fine tuning } both imply under $p(y|z,x)$

Weak gradient signal

Gradient to retriever depends on $p(y|z,x)$

Most docs $\to$ bad $\to$ noisy gradients

Signal from good docs $\to$ diluted

Credit assignment problem

Model must figure which doc caused good prediction

But supervision only not on z

Injecting Inductive Biases into Pre-training

$\to$ Salient span Masking

$\to$ Null document

$\to$ Prohibiting trivial retrievals

.! pretraining corpus & knowledge corpus

meaningful retrievals

implicit supervision →

$f(x,z)$ relevance score that is assigned for document z

analyse the gradient w.r.t params of the Knowledge retriever θ

$$\nabla \log p(y|x) = \sum_{z \in Z} r(z) \nabla f(x,z)$$

$$r(z) = \left[\frac{p(y|z,x)}{p(y|x)} - 1\right] p(z|x)$$

→ prob of predicting correct output

how good the doc is compared to average

expected value of $p(y|z,x)$ when randomly sampling a document from $p(z|x)$

if $r(z)$ +ve → inc $f(z,x)$

if $r(z)$ −ve → dec $f(z,x)$

$r(z)$ is +ve if & only if $p(y|z,x) > p(y|x)$

Index becomes stale as retriever params θ get updated

$\text{Embed}_z(z) \neq$ current model

Creates a mismatch between → retrieval scores and actual model distributions

Asynch refresh = practical fix

In gradient discussion

if the document is good → increase its score

$p(y|z,x) > p(y|x) \Rightarrow r(z) > 0$

& vice versa

This is advantage function RL-like

advantage = reward − baseline

$p(z|x)$ = weighting $r(z)$ = scaled advantage

$f(x,z) \uparrow$ if z helps prediction

Final System View

Representation

shared embedding space for x & z

Objective

$$\max \log \sum p(y|z,x)\, p(z|x)$$

Information flow

$x \to p(z|x) \to z \to p(y|z,x)$

Optimization Geometry

latent variable model

noisy gradient through marginalization

RL like credit assignment

Systems Engineering

→ approximate retrieval (MIPS)

→ stale index problem

→ async refresh

→ top k truncation

Approximation

$$\approx \log \sum_{z \in \text{top}k} p(y|z,x)\, p(z|x)$$

Top k approximation?

Most docs → negligible probability

So restrict top

↳ This is not just efficiency

It also changes gradient behavior

Only top k docs get gradients

Others → zero signal

MIPS + INDEX

↳ inner product ≈ ranking proxy

↳ pre compute doc embeddings → Initialization

↳ store in index

↳ retrieve efficiently

Problem

Retriever can cheat

pick trivial matches

ignore retrieval

collapse behavior

fixes

force knowledge dependent tokens

allow no-retrival option

remove shortcut solutions

System → Two time scale learning

Retriever → slow, indirect learning

Encoder → fast, direct learning

} this creates instability, dependency on good initialization, need for heuristics (top k, refresh etc)

Value of k whether small or big induces bias-variance trade off discussion

K controls exploration v exploitation

★ Log outside v inside sum

$$\log \sum p(y|z,x)\, p(z|x)$$

isn't equal to $\sum_z \log p(y|z,x)$

Retriever defines $p(z|x)$

it's a distb^n, not a hard selection

Training is not decomposable per document

Documents compete inside the sum

Retriever decides what info is seen

Encoder decides how useful that info is

Bad docs ↓ dilute total probability

Good docs ↑ increase total probability

This, s why gradients become relative not absolute

REALM is hard because optimization happens over a log of a mixture distb^n not

.! pretraining corpus & Knowledge corpus are same?

There exists a z which is too informative

Encoder can predict y by looking at unmasked version of x in z

This results in large positive gradients for $p(z|x)$

if this occurs too often Knowledge retriever learns to look for exact string matches between x & z and doesn't capture other forms of relevance

Initialization

beginning of training → bad embedding

This makes encoder learn to ignore earlier retrieved documents

→ no meaningful gradient

They avoid this cold start problem using Inverse Cloze Task (ICT), where given a sentence the model is trained to retrieve the document it came from

→ stale index problem
→ async refresh
→ top k truncation

Feedback loop

Better retriever → better context → better encoder
Better encoder → better signal → better retriever

The embedding space learns

$x^T z \uparrow$ iff z helps predict masked tokens

Not semantic similarity in general

But task conditioned relevance

REALM → external Memory system

not absolute

REALM is hard because optimization happens over a log of a mixture distribution not independent terms

Why inner product?

$f(x, z) = x^T z$

fast (critical for MIPS)

Enables ANN Training

Compatible with vector search systems

Alternatives?

Cross attention scoring
Learned scoring networks
} too slow for millions of docs

Why MLM specifically?

provides dense supervision

No need for labeled QA data

forces retrieval of world knowledge tokens

MLM gives signal at token level